CERGY PARIS UNIVERSITÉ

Rapport de Projet

Novembre 2025 - Janvier 2026

Application Java pour l'analyse des métadonnées audio et la création de playlists

---

Réalisé par

DEHOUCHE Ines
AHFIR Meriem

groupe : A-06

Année universitaire 2025/2026

1. Introduction **3**
1. 1 Contexte du projet 3
1.2 Objectifs du projet 3
2. Architecture du projet **5**
2.1 Structure des packages 5
2.1.1 Package metadonnees 5
2.1.2 Package exploration 5
2.1.3 Package playlists 5
2.1.4 Package cli 6
2.1.5 Package gui 6
2.2 Diagramme de classes UML 7
3. Diagramme de cas d'utilisation **7**
3.1 Acteurs 8
3.2 Cas d'utilisation principaux 8
4. Planning et répartition des tâches **9**
4.1 Diagramme de GANTT 9
5. Aspects techniques et spécificités de réalisation **9**
5.1 Bibliothèques externes utilisées 9
5.2 Exploration récursive des dossiers 10
5.3 Validation des fichiers MP3 11
5.4 Gestion des formats de playlist 11
5.5 Architecture de l'interface graphique 12
5.6 Lecteur audio intégré 13
5.7 Gestion des playlists utilisateur 13
6. Interface en ligne de commande (CLI) **13**
6.1 Options disponibles 13
6.2 Exemples d'utilisation 13
6.3 Gestion des erreurs 14
7. Niveau d'aboutissement **14**
7.1 Fonctionnalités complètes 14
7.2 Améliorations apportées 15
8. Analyse critique **15**
8.1 Points forts 15
8.2 Points faibles et limitations 16
8.3 Difficultés rencontrées 16
9. Conclusion **16**
Annexes 17
Annexe A : Bibliothèques externes 17
Annexe B : captures d'écran représentatives de l'interface GUI 17
Annexe C : verification MIME 18
Annexe D : méthode itérative 18
Annexe E : gestion de la musique (barre de progression) 19
Annexe F : Organisation et arborescence du projet ProjetMP3 19

1. Introduction

1. 1 Contexte du projet

Ce projet a été réalisé dans le cadre du module de Programmation Orientée Objet et Java de la Licence 2 Informatique. Il s'inscrit dans une démarche pédagogique visant à consolider les connaissances théoriques acquises en programmation orientée objet à travers la conception et le développement d'une application logicielle complète en Java.

Le projet consiste à gérer des fichiers audio MP3 et des playlists. Ce domaine est très concret et largement utilisé. Il permet de traiter plusieurs problèmes importants du développement logiciel. Le projet utilise des notions comme l'exploration du système de fichiers, la manipulation de données multimédias, l'extraction de métadonnées, ainsi que la structuration d'une application selon une architecture modulaire et maintenable.

De plus, avoir deux modes d'interaction, comme la console et la version graphique, aide à comparer différentes façons d'utiliser et de concevoir des interfaces utilisateur.

1.2 Objectifs du projet

L'objectif principal de ce projet est de développer une application Java permettant la manipulation de fichiers MP3 et la gestion de playlists, tout en respectant les principes de la programmation orientée objet et les bonnes pratiques de conception logicielle.

Plus précisément, l'application doit permettre :

_ L'extraction et l'affichage des métadonnées ID3 des fichiers MP3 (titre, artiste, album, durée) ;

_ L'exploration récursive d'une arborescence de dossiers, avec un filtrage des fichiers MP3 valides ;

_ la création et la gestion de playlists personnalisées ;

_ L'exportation des playlists est possible vers plusieurs formats standards, notamment le XSPF, le JSPF et le M3U8.

Il existe deux modes d'utilisation complémentaires qui sont mis à disposition. Ces modes peuvent être utilisés de manière complémentaire. Le premier mode d'utilisation complémentaire est mis à disposition pour répondre à certains besoins. Le deuxième mode d'utilisation complémentaire est également mis à disposition pour répondre à d'autres besoins. Les deux modes d'utilisation complémentaires peuvent être utilisés de différentes manières pour atteindre des objectifs spécifiques. La mise à disposition de ces deux modes d'utilisation complémentaires offre plus de flexibilité et de possibilités.

_ un mode console (CLI) reposant sur des arguments en ligne de commande,

_ un mode graphique (GUI) développé à l'aide de JavaFX ;

la lecture audio des morceaux, accompagnée d'une barre de progression, afin d'améliorer l'expérience utilisateur.

2. Architecture du projet

2.1 Structure des packages

L'application a été créée de manière à ce que chaque partie ait ses propres tâches. Cela signifie que les différentes parties de l'application sont séparées de façon claire. Cette façon de faire rend les choses plus faciles pour mettre à jour le code, le modifier et comprendre comment fonctionne le projet dans son ensemble.

Le projet est structuré autour de cinq packages principaux, chacun correspondant à un rôle fonctionnel précis.

2.1.1 Package metadonnees

Ce package est dédié à la manipulation des fichiers MP3 et à l'extraction de leurs métadonnées. Il centralise toute la logique liée aux informations ID3, indépendamment des interfaces utilisateur.

FichierMP3.java : représente un fichier MP3 et assure sa validation, notamment par la vérification de l'extension et du type MIME.

Metadonnee.java : encapsule les informations issues des métadonnées ID3, telles que le titre, l'artiste, l'album, la durée, le genre, l'année, ainsi que l'image de couverture lorsque celle-ci est disponible.

2.1.2 Package exploration

Ce package est responsable de l'exploration du système de fichiers. Il permet d'identifier automatiquement les fichiers MP3 présents dans une arborescence donnée.

ExplorateurDossier.java : assure le parcours récursif des dossiers et filtre les fichiers afin de ne conserver que les fichiers MP3 valides.

2.1.3 Package playlists

Ce package regroupe l'ensemble de la logique métier liée aux playlists. Il a été conçu de manière extensible, afin de faciliter l'ajout de nouveaux formats à l'avenir.

Playlist.java : modèle de données représentant une playlist et la collection de morceaux associée.

PlaylistExporter.java : classe abstraite définissant le contrat commun pour l'export des playlists.

ExporterXSPF.java, **ExporterJSPF.java** et **ExporterM3U8.java :** sont des implémentations concrètes qui permettent d'exporter les playlists vers les formats XSPF, JSPF et M3U8 correspondants.

PlaylistImporter.java : interface définissant le contrat pour l'import de playlists.

ImporterXSPF.java, **ImporterJSPF.java** et **ImporterM3U8.java :** sont les implémentations concrètes pour l'importation des différents formats de fichiers pris en charge.

2.1.4 Package cli

Ce package est utilisé pour l'interface en ligne de commande de l'application. Il permet d'utiliser l'application rapidement et de manière scriptable.

Le fichier **CLI.java** est responsable de l'analyse des arguments passés en ligne de commande. Il orchestre également l'exécution des différentes fonctionnalités lorsque le programme est exécuté en mode console.

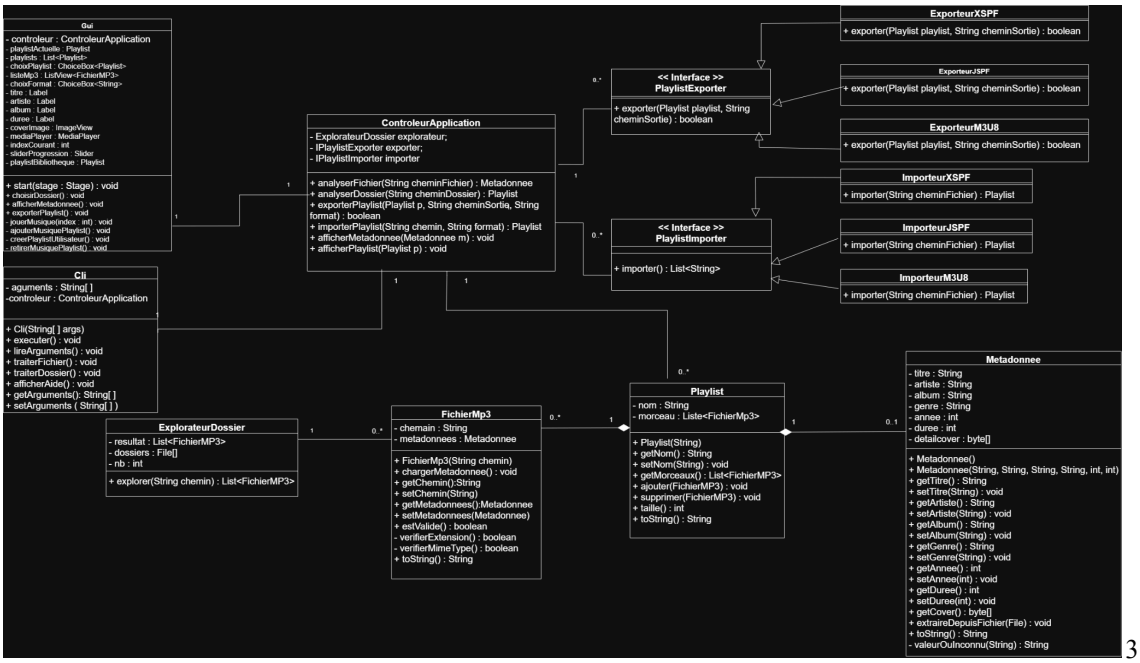
2.1.5 Package gui

Ce package est utilisé pour l'interface graphique. Il a été créé avec JavaFX. Son but est de permettre l'interaction avec l'utilisateur final et de présenter les données de manière appropriée.

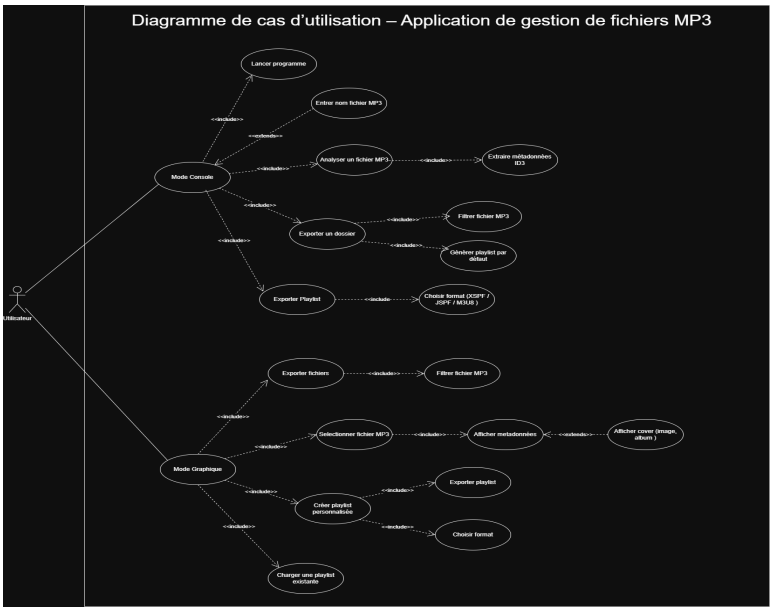
GUI.java : point d'entrée de l'application graphique, responsable de l'initialisation de l'interface et des composants visuels.

ControleurApplication.java : contrôleur central assurant la communication entre l'interface graphique et la logique métier de l'application.

2.2 Diagramme de classes UML



3. Diagramme de cas d'utilisation



3.1 Acteurs

Utilisateur : Personne utilisant l'application pour gérer sa bibliothèque musicale

3.2 Cas d'utilisation principaux

Mode Console :

_ Analyser un fichier MP3 et afficher ses métadonnées

_ Explorer un dossier et générer une playlist par défaut

_ Exporter une playlist dans un format spécifique

Mode Graphique :

_ Parcourir un dossier et visualiser tous les fichiers MP3

_ Consulter les métadonnées détaillées (y compris la pochette d'album)

_ Créer des playlists personnalisées

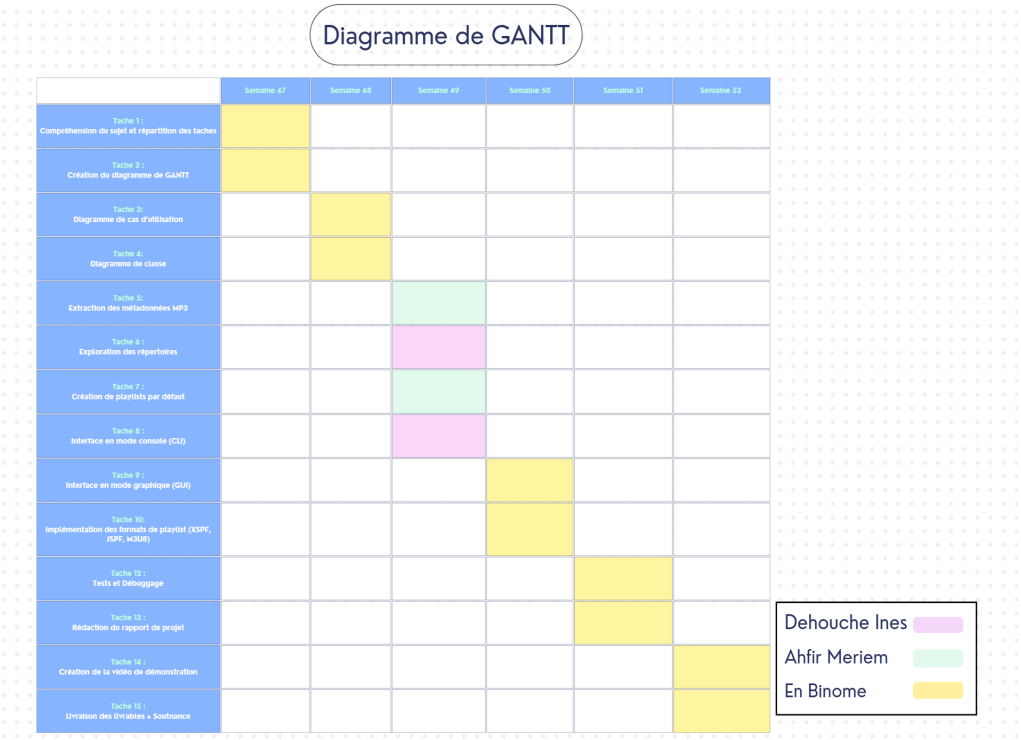
_ Ajouter/retirer des morceaux d'une playlist

_ Exporter une playlist au format choisi

_ Lire un morceau avec contrôle de la progression

4. Planning et répartition des tâches

4.1 Diagramme de GANTT



5. Aspects techniques et spécificités de réalisation

5.1 Bibliothèques externes utilisées

Pour répondre aux exigences du projet, nous avons intégré plusieurs bibliothèques externes.

JAudioTagger (org.jaudiotagger) :

Cette bibliothèque est utilisée pour la gestion des métadonnées des fichiers MP3. Elle permet notamment :
la lecture des tags ID3 (titre, artiste, album, genre, année, durée), l'extraction de l'image de couverture associée à l'album, une prise en charge robuste des différentes versions du standard ID3.
L'utilisation de JAudioTagger garantit une compatibilité élevée avec la majorité des fichiers MP3 rencontrés et simplifie considérablement le traitement des métadonnées.

JavaFX :

JavaFX est employé pour le développement de l'interface graphique. Il offre un framework moderne et bien adapté aux applications interactives.
Les principaux composants utilisés sont les suivants :

ListView pour l'affichage des fichiers MP3,
Un ChoiceBox est utilisé pour sélectionner les playlists et les formats d'export.

Le BorderPane est utilisé pour l'organisation générale de l'interface. Il s'agit d'un conteneur de layout qui permet de diviser l'interface en cinq régions principales. Le BorderPane est très utile pour créer des interfaces utilisateur complexes. Il permet de placer des éléments dans les régions du haut, du bas, de gauche, de droite et du centre. Cela aide à organiser les différents composants de l'interface de manière logique et intuitive pour l'utilisateur. Le BorderPane est un choix populaire pour l'organisation générale de l'interface utilisateur dans de nombreux types d'applications.

Un slider pour la progression de la lecture audio est très utile. Il permet à l'utilisateur de voir facilement où il en est dans l'audio et de naviguer rapidement vers une partie spécifique. Le slider pour la progression de la lecture audio est un élément essentiel dans les lecteurs audio en ligne. Il aide l'utilisateur à contrôler facilement la lecture audio et à gérer son temps de manière efficace. Le slider pour la progression de la lecture audio est donc un outil très pratique pour les utilisateurs.

L'ImageView est un composant utile pour afficher des images, comme la pochette d'un album. Il permet de charger et d'afficher l'image de la pochette d'album de manière efficace.

La classe MediaPlayer de JavaFX est utilisée pour offrir une lecture audio native et fluide.

5.2 Exploration récursive des dossiers

L'exploration des dossiers est implémentée dans la classe ExplorateurDossier.
Plutôt qu'une approche récursive classique, une approche itérative a été retenue, basée sur un tableau de dossiers à explorer :

```
File[] dossiers = new File[500];
int nb = 0;
```

L'exploration est réalisée de manière **itérative**, à l'aide d'un tableau qui stocke les dossiers à explorer. Le dossier racine est ajouté en premier, puis chaque sous-dossier rencontré est ajouté à ce tableau et exploré à son tour. Cette méthode évite l'utilisation de la récursion et limite ainsi les risques de dépassement de pile.

Lors du parcours, chaque élément est analysé :

_ si c'est un dossier, il est ajouté à la liste des dossiers à explorer,

_ si c'est un fichier avec l'extension .mp3 , il est converti en objet FichierMP3 et ajouté à la liste des résultats.

À la fin de l'exploration, la méthode retourne la liste de tous les fichiers MP3 trouvés, qui est ensuite utilisée pour créer la playlist bibliothèque de l'application.

5.3 Validation des fichiers MP3

Une double validation est implémentée dans la classe FichierMP3 afin de garantir la fiabilité des fichiers traités :
_ Vérification de l'extension
_ Le fichier doit posséder l'extension .mp3, sans sensibilité à la casse.
_ Vérification du type MIME

Le type MIME est contrôlé via la méthode Files.probeContentType() et doit correspondre à audio/mpeg.
Cette double vérification permet d'éviter notamment les fichiers qui ont simplement été renommés, et elle assure une compatibilité correcte avec la bibliothèque JAudioTagger.

5.4 Gestion des formats de playlist

L'application prend en charge trois formats de playlists distincts, ce qui offre une certaine flexibilité.

Format XSPF (XML) :
_ Format structuré basé sur XML,
_ Validation effectuée à l'aide du validateur en ligne officiel (validator.xspf.org),
_ Échappement systématique des caractères spéciaux XML (&, <, >, ", '),
_ Support complet des métadonnées (titre, artiste, album, durée).

Format JSPF (JSON):
Équivalent JSON du format XSPF, pertmet la gestion des caractères spéciaux et des guillemets.

Format M3U8 (texte UTF-8) :
_ Format simple et très compatible,
_ Utilisation des lignes #EXTINF pour stocker les métadonnées des pistes,
_ Chemins absolus afin d'assurer une compatibilité multi-plateforme.

5.5 Architecture de l'interface graphique

L'interface graphique repose sur un BorderPane, permettant une organisation claire des différentes zones :

Zone TOP :
La barre d'outils principale comprend les éléments suivants :
_ un bouton de sélection de dossier,
_ une ChoiceBox pour la playlist active,
_ un bouton de création de playlist.

Zone LEFT :
Liste des fichiers MP3 via un ListView<FichierMP3>, avec affichage formaté sous la forme
(Artiste – Titre).

Zone CENTER :
Panneau d'affichage des métadonnées :titre, artiste, album, durée, image de couverture affichée dans un ImageView et un Slider de progression de la lecture.

Zone BOTTOM :
Contrôles de lecture et gestion des playlists :boutons précédent, lecture, pause, suivant, ajout et retrait de morceaux, choix du format d'export et bouton d'exportation.

5.6 Lecteur audio intégré

Un lecteur audio est intégré à l'interface graphique. Les fonctionnalités suivantes sont implémentées :
lecture, pause, morceau précédent et suivant, synchronisation de la barre de progression avec la lecture, déplacement manuel dans le morceau via le slider, sélection automatique du morceau en cours dans la liste et aussi la gestion de la fin de lecture.
La progression est mise à jour dynamiquement grâce à l'écoute de la propriété currentTimeProperty du MediaPlayer.

5.7 Gestion des playlists utilisateur

Deux types de playlists sont distingués :
_ Playlist bibliothèque : générée automatiquement lors de l'exploration d'un dossier.
_ Playlists utilisateur : créées manuellement et alimentées par la sélection de morceaux.

Les morceaux sont référencés sans duplication physique des fichiers, garantissant une gestion efficace de la mémoire.

6. Interface en ligne de commande (CLI)

6.1 Options disponibles

-h, --help : affiche l'aide
-f <fichier.mp3> : analyse un fichier MP3
-d <dossier> : explore un dossier
--xspf / --jspf / --m3u8 : format d'export
-o <fichier> : fichier de sortie

6.2 Exemples d'utilisation

java -jar cli.jar -f musique.mp3
java -jar cli.jar -d ./music/
java -jar cli.jar -d ./music/ --xspf -o playlist.xspf

6.3 Gestion des erreurs

Le mode CLI implémente une validation stricte des arguments :
Les options -f et -d ne peuvent pas être utilisées ensemble, car elles sont mutuellement exclusives. Lorsqu'on utilise l'option -f, on ne peut pas utiliser l'option -d, et vice versa.
_ La présence des valeurs associées aux options est vérifiée,

_ Un format d’export est obligatoire si l’option -o est utilisée,
_ Les messages d’erreur sont affichés explicitement sur System.err.

7. Niveau d'aboutissement

7.1 Fonctionnalités complètes

Extraction de métadonnées MP3

_ Tous les champs ID3 supportés (titre, artiste, album, genre, année, durée).

_ Extraction de la pochette d'album.

Exploration de dossiers

_ Parcours récursif complet de l'arborescence.

_ Filtrage par extension et type MIME.

Gestion de playlists

_ Création, modification, suppression.

_ Support des trois formats (XSPF, JSPF, M3U8).

_ Validation XSPF testée sur validator.xspf.org.

Interface CLI

_ Toutes les options implémentées.

_ Gestion robuste des arguments.

_ Affichage formaté des résultats.

Interface GUI

_ Navigation fluide entre playlists.

_ Affichage complet des métadonnées.

_ Gestion visuelle des pochettes d'album.

Lecteur audio (bonus)

_ Lecture avec contrôles complets.

_ Barre de progression interactive.

7.2 Améliorations apportées

_ **Lecteur audio intégré** avec barre de progression.

_ **Gestion multi-playlists** dans l'interface graphique.

_ **Affichage des pochettes d'album** avec image par défaut.

_ **Validation robuste** des fichiers MP3 (extension + MIME).

_ **Interface utilisateur moderne** avec émojis et layout intuitif.

8. Analyse critique

8.1 Points forts

Architecture modulaire

_ Séparation claire des responsabilités (packages dédiés).

_ Facilité d'extension (ajout de nouveaux formats d'export/import).

_ Réutilisation du code entre CLI et GUI via le ControleurApplication

Sécurité de fonctionnement

_ Double validation des fichiers MP3.

_ Gestion des erreurs.

_ Métadonnées par défaut en cas d'absence d'informations concernant le fichier MP3 (mettre des valeurs par défaut "inconnu")

Expérience utilisateur

_ Interface graphique intuitive et moderne.

_ Lecteur audio fonctionnel et réactif.

_ Affichage visuel des pochettes d'album.

Respect des standards

_ Fichiers XSPF validés sur validator.xspf.org.

_ Formats JSON et M3U8 conformes aux spécifications.

_ Encodage UTF-8 pour M3U8.

8.2 Points faibles et limitations

Gestion de la mémoire

_ Tableau fixe de 500 dossiers dans ExplorateurDossier .

_ Chargement de toutes les métadonnées en mémoire.

Import de playlists

_ Les classes ImporterXSPF, ImporterJSPF, ImporterM3U8 sont implémentées mais non intégrées dans l'interface graphique

8.3 Difficultés rencontrées

Synchronisation de la barre de progression Le slider de progression posait un problème de boucle infinie : la mise à jour automatique déclenchait l'événement valueChanging qui redéplaçait la musique. Solution trouvée avec la vérification !sliderProgression.isValueChanging().

Gestion des pochettes d'album absentes Certains MP3 n'ont pas de pochette intégrée. Nous avons implémenté une image par défaut.

Échappement des caractères spéciaux Les exports XSPF et JSPF. Nécessitent un échappement rigoureux pour éviter les fichiers malformés. Des tests ont été nécessaires.

9. Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble des concepts de programmation orientée objet vus en cours : héritage, polymorphisme, interfaces, classes abstraites, encapsulation. L'utilisation de JavaFX a également été l'occasion de découvrir le développement d'interfaces graphiques modernes.

Le résultat final est une application complète et fonctionnelle qui dépasse les exigences minimales du cahier des charges, notamment avec l'ajout d'un lecteur audio intégré et la gestion visuelle des pochettes d'album. Malgré quelques limitations identifiées, nous sommes satisfaits de la qualité d'expérience que va offrir notre application à l'utilisateur.

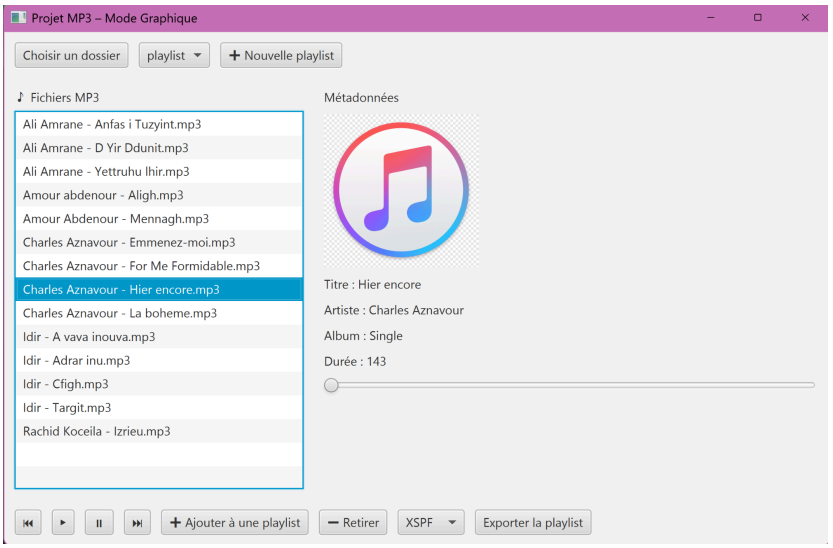
Annexes

Annexe A : Bibliothèques externes

_ **JAudioTagger 3.0.1** : http://www.jthink.net/jaudiotagger/

_ **JavaFX 21** : https://openjfx.io/

Annexe B : captures d'écran représentatives de l'interface GUI



Annexe C : verification MIME

```
    //verifie si le fichier est un MP3 valide (extension + type MIME)
    private boolean verifierExtension() {
        return chemin.toLowerCase().endsWith(".mp3");
    }

    private boolean verifierMimeType() {
        try {
            File fichier = new File(chemin);
            String mimeType = Files.probeContentType(fichier.toPath());

            if (mimeType == null) {
                return false;
            }

            return mimeType.equals("audio/mpeg");

        } catch (IOException e) {
            return false;
        }
    }
```

Annexe D : méthode itérative

```
    //parcourt le dossier racine et tous ses sous-dossiers pour recuperer tous les fichiers MP3.
     public List<FichierMP3> explorer(String chemin) {
        dossiers[nb]=new File(chemin); //ajouter le dossier racine au tableau
        nb++;
        int i=0;
        while (i < nb) {
            File dossierActuel=dossiers[i]; //prend le dossier courant
            i++;
            File[] fichiers =dossierActuel.listFiles(); //liste tous les fichiers et sous dossiers
            if (fichiers == null) {
                continue; //si le dossier est vide ou inaccessible on passe au suivant
            }
            for (File f : fichiers) {
                if (f.isDirectory()) {
                    //si c'est un sous dossier on l'ajoute au tableau pour l'explorer plus tard
                    if (nb < dossiers.length){
                        dossiers[nb]=f;
                        nb++;
                    }
                }
                else{
                    //si c'est un fichier MP3 on le transforme en FichierMp3 et on l'ajoute a la liste
                    String nom=f.getName().toLowerCase();
                    if (nom.endsWith(".mp3")) {
                        FichierMP3 mp3=new FichierMP3(f.getAbsolutePath());
                        resultat.add(mp3);
                    }
                }
            }
        }
        return resultat; //retourne la liste finale des fichiers MP3 trouves
    }
}
```

Annexe E : gestion de la musique (barre de progression)

```
 private void jouerMusique(int index) {  //joue la musique à l'index donné et gere la barre de progression
     if (playlistActuelle == null) return;
     if (index < 0 || index >= listeMp3.getItems().size()) return;
    //arret de la musique en cours
     if (mediaPlayer != null) {
         mediaPlayer.stop();
     }
     FichierMP3 mp3 = listeMp3.getItems().get(index);
     File fichier = new File(mp3.getChemin());
     Media media = new Media(fichier.toURI().toString());
     mediaPlayer = new MediaPlayer(media);
     mediaPlayer.play();
     indexCourant = index;
     listeMp3.getSelectionModel().select(index);
     //mise à jour du slider en fonction de l'avancement de la musique
     mediaPlayer.currentTimeProperty().addListener((obs, oldTime, newTime) -> {
         if (!sliderProgression.isValueChanging()) { //eviter les conflits quand l'utilisateur bouge le slider
             double progress = newTime.toSeconds() / mediaPlayer.getTotalDuration().toSeconds() * 100;
             sliderProgression.setValue(progress);
         }
     });
     //deplacement de la musique quand l'utilisateur bouge le slider
     sliderProgression.valueChangingProperty().addListener((obs, wasChanging, isChanging) -> {
         if (!isChanging) { // l'utilisateur a fini de bouger le slider
             double position = sliderProgression.getValue() / 100 * mediaPlayer.getTotalDuration().toSeconds();
             mediaPlayer.seek(javafx.util.Duration.seconds(position));
         }
     });
     sliderProgression.setOnMouseReleased(event -> { // pour un clic direct sur le slider
         double position = sliderProgression.getValue() / 100 * mediaPlayer.getTotalDuration().toSeconds();
         mediaPlayer.seek(javafx.util.Duration.seconds(position));
     });
 }
```

Annexe F : Organisation et arborescence du projet ProjetMP3

```
ProjetMP3/
├── src/
│   ├── cli/
│   │   └── CLI.java
│   ├── exploration/
│   │   └── ExplorateurDossier.java
│   ├── gui/
│   │   ├── ControleurApplication.java
│   │   └── GUI.java
│   ├── metadonnees/
│   │   ├── FichierMP3.java
│   │   └── Metadonnee.java
│   └── playlists/
│       ├── ExporterJSPF.java
│       ├── ExporterM3U8.java
│       ├── ExporterXSPF.java
│       ├── ImporterJSPF.java
│       ├── ImporterM3U8.java
│       ├── ImporterXSPF.java
│       ├── Playlist.java
│       ├── PlaylistExporter.java
│       └── PlaylistImporter.java
└── resources/
    └── cover_default.png
```